13.2.1 Django Extensions

The Django Extensions (https://pypi.org/project/django-extensions/) library is one of the most popular tool kits out there. Installation is simple: run `python -m pip install django-extensions`, then add `"django_extensions"` to `INSTALLED_APPS`. This library is quite large and is broken down by feature area, with extensions for the following:

- Commands
- Signals
- Debugging
- Fields
- Job scheduling
- Models
- Permissions
- Validators

Even with that extensive list, there is still another section called Utilities, the miscellaneous area of the ultimate miscellaneous tool kit. There is way too much in here to cover it all in detail, so I'll highlight a couple of key items.

Inside the Commands section, you'll find `shell_plus`: a management command that adds features to Django's `shell`. The key differentiator here is that it pre-imports many of the modules, classes, and functions you are likely to use when interacting with a Django shell, including `settings`, `reverse`, `Q`, and more. You can configure what gets imported, allowing you to include your own frequently used models. This can save a lot of typing.

Another useful command is `validate_templates`. This checks for syntax errors in any of your template files, allowing you to verify them first, rather than catching the error at run time. This command can be particularly useful as part of your continuous integration process, or as a commit hook, disallowing invalid templates in your repo.

The Fields section of the Extensions library has new Django Fields for your models. It includes fields for automatically generating a slug, creating random values, and using timestamps to track object creation and modification as well as a short UUID container. Along with the fields are some premade `Model` classes. When you inherit from one of them, you automatically get collections of fields that work together. For example, the `ActivatorModel` is an abstract model class that has a status field and two date fields. The status field indicates whether the object is "active," while the date fields track when an object was made active or inactive. Using this `Model`, you can quickly query only active items. This can be useful for things like blog posts, allowing you, for example, to hide a post that isn't ready for launch yet.

13.2.2 Django Awl

Similar to Django Extensions is Django Awl (https://pypi.org/project/django-awl/). Full disclosure, this is one of my own extensions. On one hand, you can mock the hubris of me including it here; on the other hand, I'm open to pull requests, so if